

**Problem # 1:** Multiple choice question. [12 x 2pts]

- 1) Which of the following statement is **Incorrect**?
 - A) Once the linear queue becomes full, we cannot insert the next element even if there is a space in front of the queue
 - B) Overwrite cannot be done in a circular queue
 - C) Circular Queue is a linear data structure in which the operations are performed based on FIFO (First In First Out) principle
 - D) Circular Queue is also called 'Ring Buffer'
- 2) Which of the following is **Correct** about the stack?
 - A) push() operation inserts an element into the stack
 - B) If top = -1, then the stack is empty
 - C) pop() operation removes an element from the stack
 - D) All of the above
- 3) Which of the following is **False** about a doubly linked list?
 - A) We can navigate in both directions
 - B) It requires more space than a singly linked list
 - C) Implementing a doubly linked list is easier than a singly linked list
 - D) None of the above
- 4) The conditions for the queue to be a circular queue are -----
 - A) rear = size-1
 - B) Front = 0
 - C) (A) and (B)
 - D) None of the above
- 5) Class and function templates allow the programmer to write one code for all data types and help in generic programming.
 - A) True
 - B) False
- 6) Which of the following is useful in the queue structure?
 - A) Controlling printer jobs
 - B) Control first-in-last-out operations in the processor
 - C) Arrange data in the file
 - D) None of the above
- 7) Undo/Redo operations in Excel or Word processing programs are a real example of linear queue data structure.
 - A) True
 - B) False



8) In a circular linked list, insertion of a node requires modification of

- A) One pointer
- B) Two pointer
- C) Three pointer
- D) None of the above

9) The deletion operation in the stack is called

- A) insert
- B) push
- C) pop
- D) enqueue

10) In the linear queue data structure, the value of **Front** is increased by 1 when

- A) An element is deleted in a queue
- B) An element is traversed in a queue
- C) An element is added to a queue
- D) An element is merged in a queue

11) A linked list in which none of the nodes contains a NULL pointer is

- A) Circular linked list
- B) Singly linked list
- C) Doubly linked list
- D) None of the above

12) Which of the following information is stored in a doubly-linked list's nodes?

- A) Value of node
- B) Address of next node
- C) Address of the previous node
- D) All of the above

*** Write the correct answers to the problem # 1 in the table below:

<u>Q1</u>	<u>Q2</u>	<u>Q3</u>	<u>Q4</u>	<u>Q5</u>	<u>Q6</u>
<u>Q7</u>	<u>Q8</u>	<u>Q9</u>	<u>Q10</u>	<u>Q11</u>	<u>Q12</u>



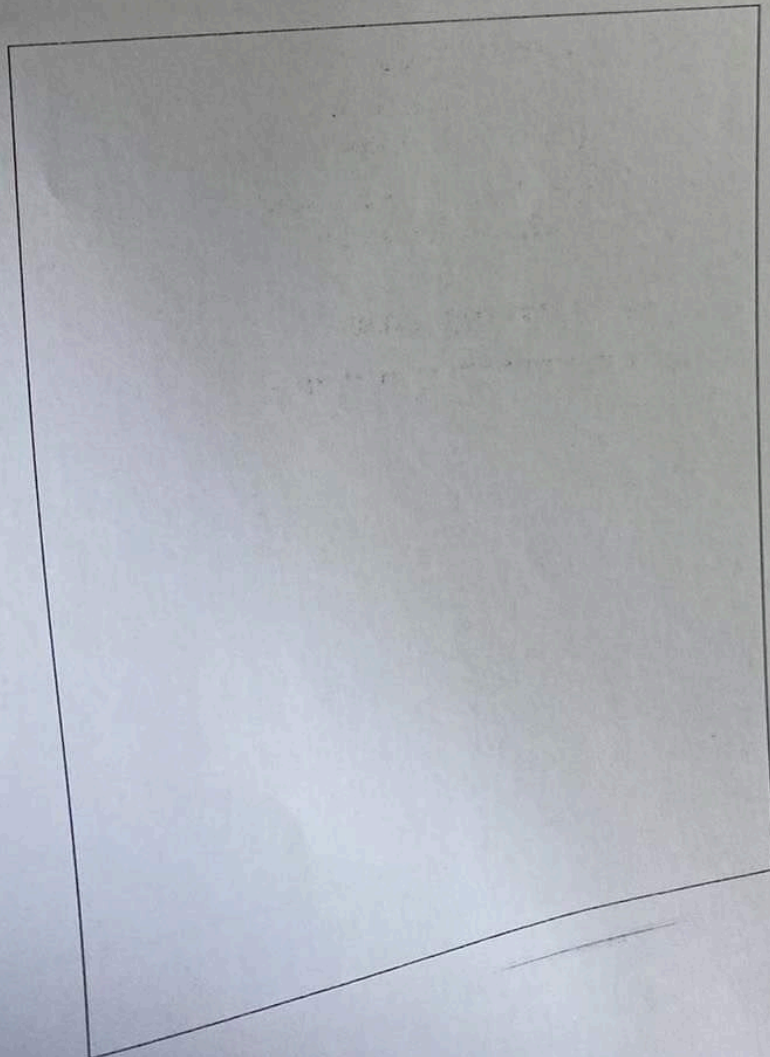
Problem # 2: What are the differences between arrays and linked lists in terms of size, memory allocation time, data access, complexity for inserting/deleting elements, memory efficiency/utilization, and exploiting the spatial locality in caches? [14 x 0.5pt]

	Arrays	Linked lists
Size (fixed or dynamic)		
Memory allocation time		
Data access (sequentially or randomly)		
The complexity of Inserting/deleting elements		
Memory efficiency/Utilization		
Required Memory space		
Exploiting the spatial locality in cache memories		

is the output of the C++ main below? [5pts]

```
int main() {  
    linkedstack s1;  
    s1.push(52);  
    s1.push(41);  
    s1.push(71);  
    s1.push(72);  
    s1.push(42);  
    s1.push(61);  
    s1.push(31);  
    s1.push(51);  
    s1.push(62);  
    s1.push(32);  
    s1.func4();  
    s1.display();  
    return 1;  
}
```

The output of Part D:





The C++ program for linkedstack in Problem # 4

```
#include <iostream>
using namespace std;

class linkedstack
{
private:
    struct node
    {
        int data;
        node *link;
    };
    node *head;
public:
    linkedstack();
    void push(int element);
    int pop ();
    void display();
    int count();
    int func1(int pos_node1, int pos_node2);
    int func2();
    int func3();

    ~linkedstack();
};

linkedstack::linkedstack()
{
    head = NULL;
}

void linkedstack::push(int element)
{
    node *q,*t;
    if( head == NULL ){
        head = new node;
        head->data = element;
        head->link = NULL;
    }
    else{
        q = head;
        while( q->link != NULL )
            q = q->link;
        t = new node;
        t->data = element;
        t->link = NULL;
        q->link = t;
    }
}
```



```
int linkedstack::pop (){
    int x;
    if( head == NULL ) // check if the stack is empty
    {
        cout<<"empty stack";
        return 0;
    }
    else // delete from the end of the stack
    {
        node *q,*r;
        q = head;
        r = q;
        while( q->link!=NULL )
        {
            r = q;
            q = q->link;
        }
        r->link=NULL;
        x=q->data;
        delete q;
        return x;
    }
}

void linkedstack::display(){
    node *q;
    for( q = head ; q != NULL ; q = q->link )
        cout<<q->data<<endl;
}

int linkedstack::count(){
    node *q;
    int c=0;
    for( q=head ; q != NULL ; q = q->link )
        c++;
    return c;
}

linkedstack::~~linkedstack(){
    node *q;
    if( head == NULL )
        return;
    while( head != NULL )
    {
        q = head->link;
        delete head;
        head = q;
    }
}
```

Problem # 3: Write a C++ program to reverse a singly ordered link list. [3pts]

```
void orderedlinklist::reverse()
```

```
{
```

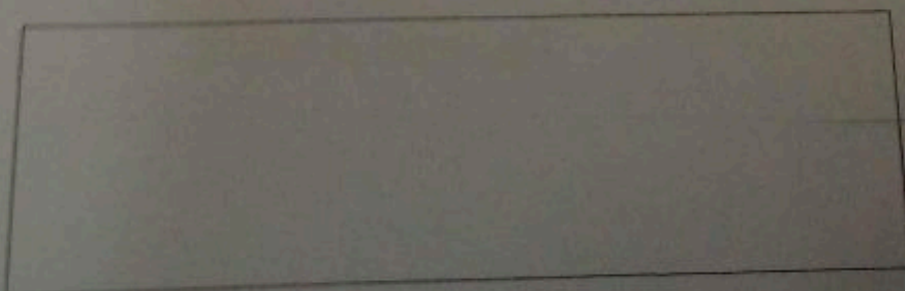
```
}
```

Problem # 4: Based on the C++ program written in the external data sheet, which represents the implementation of stack using Linked list (Linkedstack) and stores integer values, answer questions A to E: [16pts]

What is the output of the C++ main below? [2pts]

```
int main(){
    linkedstack s1;
    s1.pop();
    s1.push(12);
    s1.push(13);
    s1.push(2);
    cout<<"n"<<"The popped data item is"<<" "<<s1.pop()<<"\n";
    cout<<"The popped data item is"<<" "<<s1.pop()<<"\n";
    s1.display();
    return 1;
}
```

The output of Part A:





Problem # 3: Write a C++ program to reverse a singly ordered link list. [3pts]

```
void orderedlinklist::reverse()
{
```

```
}
```

Problem # 4: Based on the C++ program written in the external data sheet, which represents the implementation of stack using Linked list (Linkedstack) and stores integer values, answer questions A to E: [16pts]

What is the output of the C++ main below? [2pts]

```
int main(){
    linkedstack s1;
    s1.pop();
    s1.push(12);
    s1.push(13);
    s1.push(2);
```